Heart-Disease Risk Classifier as a Monitored, Containerised API

MLOps (AIMLCZG523) - Assignment 01

Jayanth Saish Pamarti | BITS Pilani WILP

Report generated 19 October 2026

Repository: https://github.com/2025ae05879/mlops-heart-disease

Deployment: Local access: API http://localhost:8080 (Kubernetes LoadBalancer), Prometheus http://localhost:9090, Grafana http://localhost:3000.

Summary

This project turns the UCI Cleveland heart-disease data into a small but complete MLOps system. A script downloads the data. An executed notebook explains it. One scikit-learn pipeline holds every preprocessing step, and three model families are tuned with stratified cross-validation and tracked in MLflow. The selected model is packaged with joblib and the MLflow format, tested with pytest, and wired into a GitHub Actions workflow. It is served by FastAPI in a Docker image, deployed with two replicas on Docker Desktop Kubernetes, and observed through structured logs, Prometheus and Grafana. The final model is logreg, chosen on CV recall: CV ROC-AUC 0.910 +/- 0.025, test ROC-AUC 0.913.

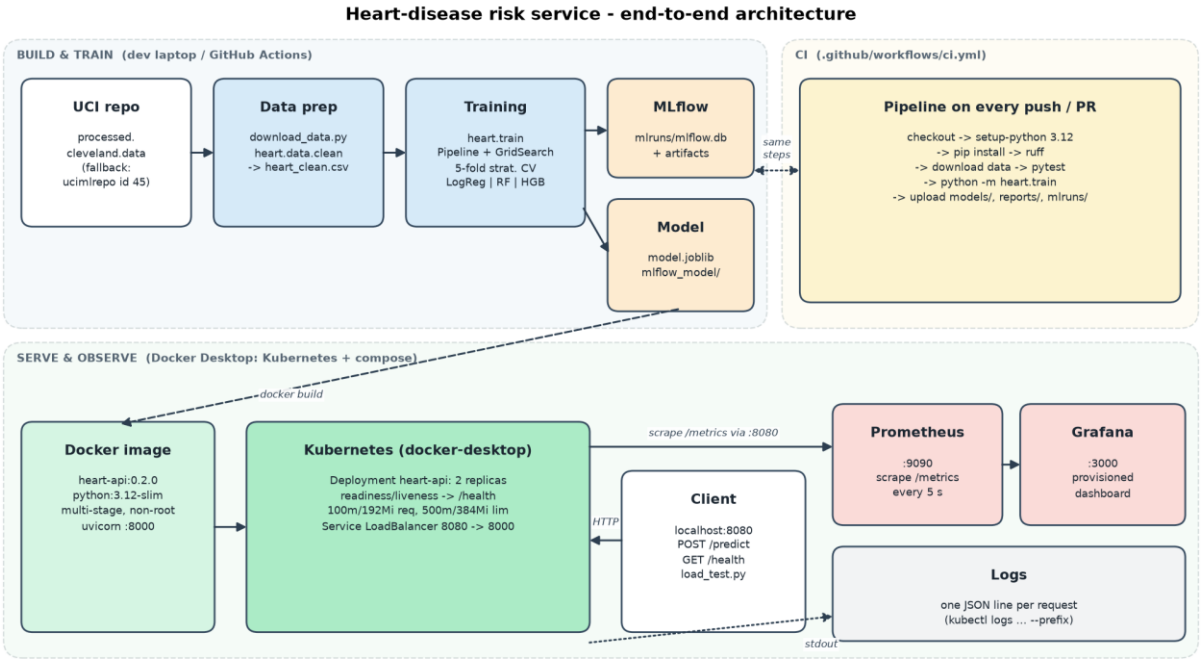


Figure 1. System architecture: build/train, CI, serving and observability.

1. Problem and data

The goal is a binary classifier that estimates whether a patient has angiographic heart disease from 13 routinely collected attributes. The intended use is screening: the model flags patients who should get a closer look, so a missed positive (false negative) costs more than an unnecessary follow-up. That asymmetry drives the model-selection rule in Section 4.

The data is processed.cleveland.data from the UCI Machine Learning Repository: 303 patients and 14 columns, with no header and '?' marking missing values. scripts/download_data.py fetches it over HTTPS, falls back to the ucimlrepo package (dataset id 45) if the URL fails, writes data/raw/cleveland_raw.csv with column names, and then writes the cleaned data/processed/heart_clean.csv. The original target `num` grades disease from 0 to 4; it is binarised as num > 0 -> 1, which gives 139 positives (46%) against 164 negatives.

Table 1. Column roles.

columns	type	treatment in the pipeline
age, trestbps, chol, thalach, oldpeak	continuous	median impute + StandardScaler
ca (vessels coloured, 0-3)	count / ordinal	median impute + StandardScaler
cp, restecg, slope, thal	nominal codes	mode impute + OneHotEncoder(ignore unknown)
sex, fbs, exang	binary flags	mode impute, passed through
pct_max_hr, st_per_mmhg	derived in pipeline	median impute + StandardScaler

2. Exploratory data analysis

The full analysis is in notebooks/01_eda.ipynb, which is executed with nbconvert so its outputs are saved. All figures are written to reports/figures/.

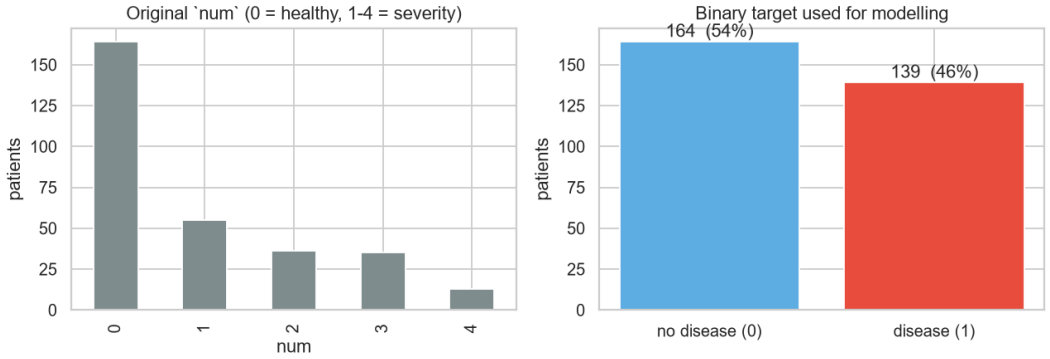


Figure 2. Original severity grades vs. the binary target. Grades 1-4 range from 13 to 55 patients each, too few to model separately.

Class balance. The classes are close to balanced, so I did not resample. Stratification at every split, plus class_weight as a tunable option, was enough.

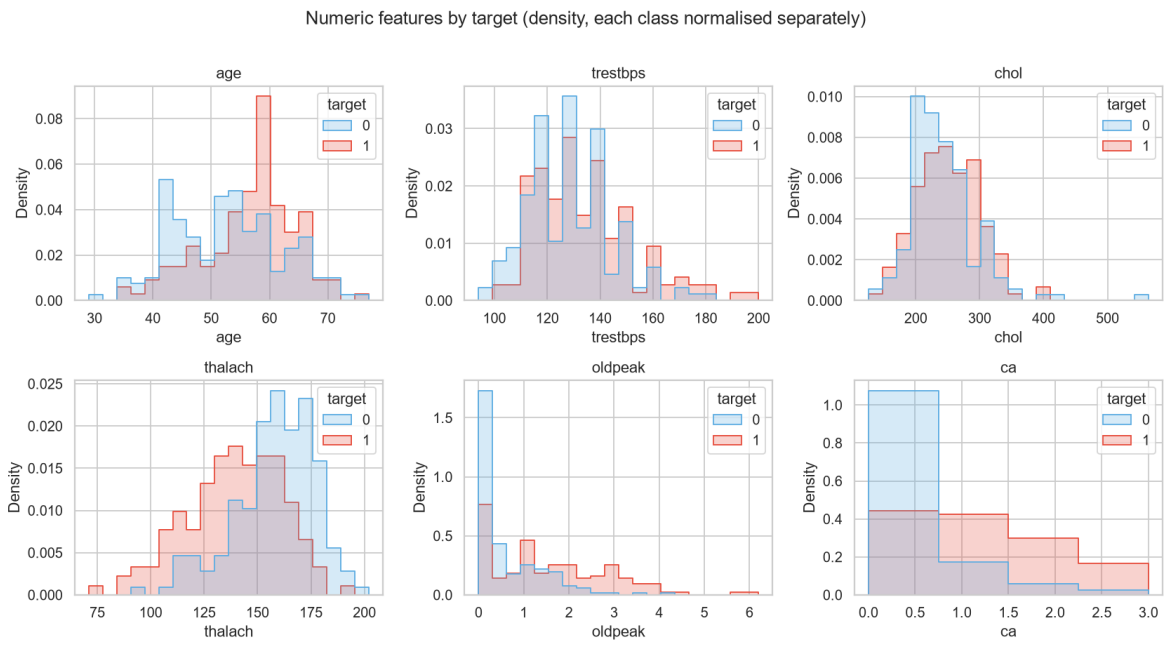


Figure 3. Numeric features by outcome (each class normalised separately).

Distributions. Peak exercise heart rate (thalach) and ST depression (oldpeak) separate the classes the most. Diseased patients peak about 19 bpm lower (medians 142 vs 161), and their median ST depression is 1.4 mm against 0.2 mm. The number of coloured vessels (ca) is similarly informative. Resting blood pressure has identical medians (130) in both groups, and cholesterol differs by only about 15 mg/dl. In this already-referred cohort, the classic risk factors matter much less than the stress-test results.

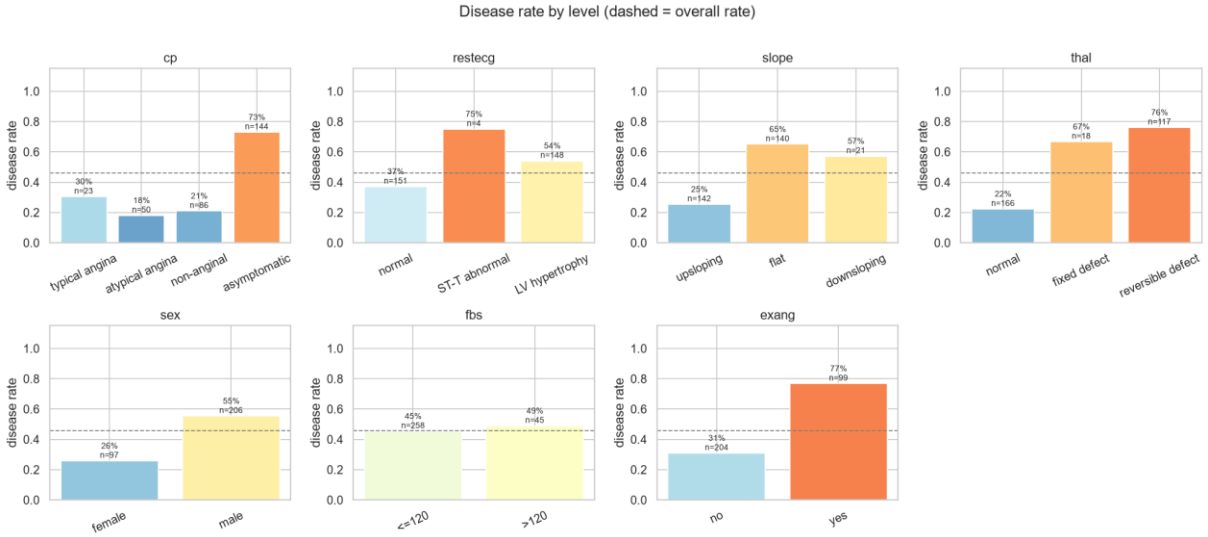


Figure 4. Disease rate per category level (dashed line = overall 46%).

Categorical levels. Asymptomatic chest pain (cp = 4) carries a 73% disease rate, while typical or atypical angina sits at 18-30%. That looks counter-intuitive, but it reflects a referral population in which silent ischaemia was found through other tests. Exercise-induced angina (77% vs 31%) and a reversible thallium defect (76%) are strong signals. Fasting blood sugar is almost flat (45% vs 49%). restecg = 1 has only 4 patients, so a CV fold can miss that level entirely, which is why the encoder ignores unknown categories.

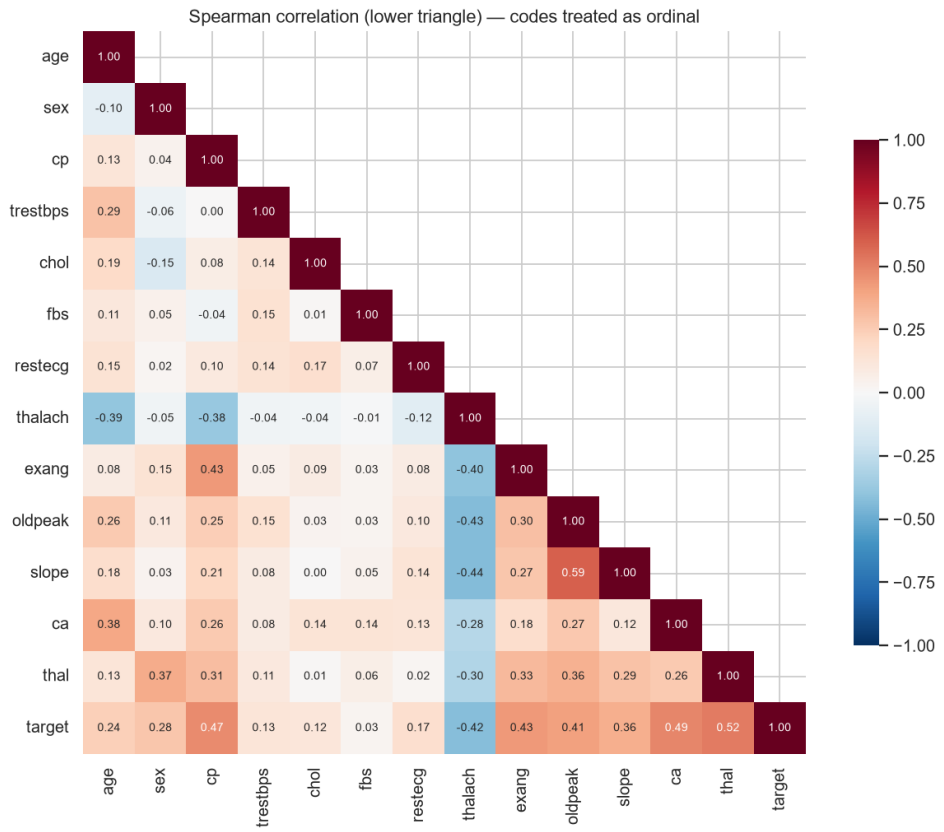


Figure 5. Spearman correlation (lower triangle).

Correlation. The strongest links to the target are thal (0.52), ca (0.49), cp (0.47), exang (0.43), thalach (-0.42) and oldpeak (0.41). Among the features themselves, the largest pair is oldpeak-slope (0.59). No pair is collinear enough to justify dropping a feature, and L2 regularisation absorbs the rest.

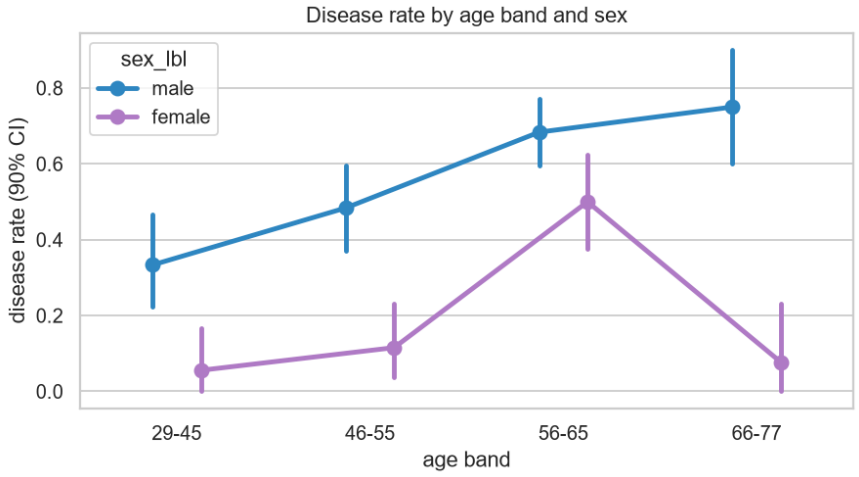


Figure 6. Disease rate by age band and sex (90% CI).

Own analysis - interactions. In men, the rate rises steadily with age, from 33% to 75%. In women it stays at 6-12% until 56-65, where it jumps to 50%. Combining chest pain with the thallium result shows a similar interaction: asymptomatic patients with a reversible defect (78 patients) reach 91% (figure cp_thal_interaction.png in the repo). These interactions are why tree ensembles were included alongside the linear baseline.

3. Cleaning and feature reasoning

- **Missing values.** Only 6 rows are incomplete: 4 missing ca and 2 missing thal. They are kept and imputed inside the pipeline (median for ca, mode for thal) rather than dropped or filled in the CSV. Dropping them would lose 2% of a small dataset. Imputing inside the pipeline fits the imputer on training folds only, so CV and test scores have no leakage, and the deployed API can accept a request without ca or thal.
- **Domain validation.** Coded columns are checked against the UCI codebook, and any value outside it becomes NaN and goes through the same imputation path. No out-of-domain values occur in the file; the check protects future inputs.
- **Encoding.** Nominal codes (cp, restecg, slope, thal) are one-hot encoded. ca is kept numeric because the disease rate rises roughly monotonically with the number of vessels. Binary flags pass through unchanged.
- **Derived features.** pct_max_hr = thalach / (220 - age) expresses peak heart rate relative to the age-predicted maximum, separating 'low because older' from 'could not reach target rate'. st_per_mmhg = oldpeak / trestbps x 100 gives linear models an ST-depression x blood-pressure interaction. Both are computed by a FunctionTransformer inside the pipeline, so callers send only the 13 raw fields. The EDA checked them honestly: pct_max_hr alone scores AUC 0.70 against 0.75 for raw thalach, so it complements thalach rather than replacing it. Neither appears in the selected model's ten largest coefficients.

4. Model development and tuning

Each candidate is a Pipeline(prep -> clf), where prep runs the derived features followed by a ColumnTransformer (impute + scale | impute + one-hot | impute binary). The data gets a stratified 80/20 hold-out split with seed 523; the 61 test rows are never used for tuning or selection. Each model is tuned by GridSearchCV with StratifiedKFold(5, shuffle=True, seed 523), scored on accuracy, precision, recall, F1 and ROC-AUC, and refit on ROC-AUC. The grids are deliberately small, because with ~240 training rows a wide grid mostly overfits the folds.

Table 2. Search spaces.

model	grid	configs x 5 folds
Logistic Regression (saga)	C {0.01-3}, l1_ratio {0, 0.5, 1}, class_weight	36
Random Forest	trees {200, 500}, max_depth {3, 5, None}, min_samples_leaf {1, 3, 5}, max_features {sqrt, 0.5}	36
HistGradientBoosting	learning_rate {0.03, 0.1}, max_depth {2, 3}, max_iter {100, 300}, l2 {0, 1}	16

Table 3. Cross-validation results on the training split (mean +/- std over 5 folds, best grid point).

model	cv_accuracy	cv_precision	cv_recall	cv_f1	cv_roc_auc
logreg	0.859 +/- 0.034	0.870 +/- 0.055	0.820 +/- 0.048	0.843 +/- 0.036	0.910 +/- 0.025
random_forest	0.814 +/- 0.057	0.852 +/- 0.092	0.730 +/- 0.044	0.785 +/- 0.058	0.889 +/- 0.045
hist_gb	0.777 +/- 0.025	0.775 +/- 0.057	0.739 +/- 0.050	0.753 +/- 0.013	0.885 +/- 0.025

Table 4. Held-out test results (n = 61, threshold 0.5) and selected hyper-parameters.

model	test_accuracy	test_precision	test_recall	test_f1	test_roc_auc	best_params
logreg	0.836	0.821	0.821	0.821	0.913	{"C": 0.3, "class_weight": null, "l1_ratio": 0.0}
random_forest	0.852	0.828	0.857	0.842	0.917	{"max_depth": 3,

						"max_features": "sqrt", "min_samples_leaf": 5, "n_estimators": 500}
hist_gb	0.82	0.793	0.821	0.807	0.921	{"l2_regularization": 1.0, "learning_rate": 0.03, "max_depth": 2, "max_iter": 300}

Selection rule. Every model whose CV ROC-AUC is within 0.02 of the best is shortlisted, and the shortlisted model with the highest CV recall wins, because recall protects against missed patients. Logreg led on both CV ROC-AUC (0.910 +/- 0.025) and CV recall (0.820 +/- 0.048), so it was selected. It is also the easiest to explain to a clinician. Random Forest shows slightly higher test recall (0.857 vs 0.821), but that is a difference of one patient out of 28 positives in a 61-row test set, and it was clearly weaker in CV. Choosing on the test set would have been selection bias.

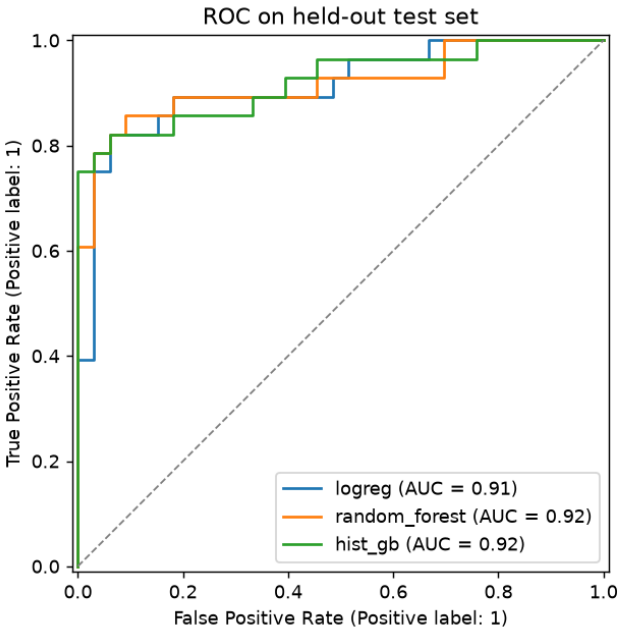


Figure 7. ROC curves of the three tuned models on the held-out test set.

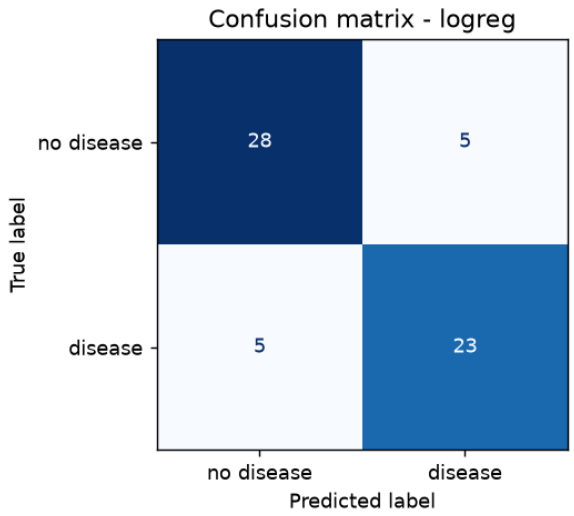


Figure 8. Confusion matrix of the selected model (test set).

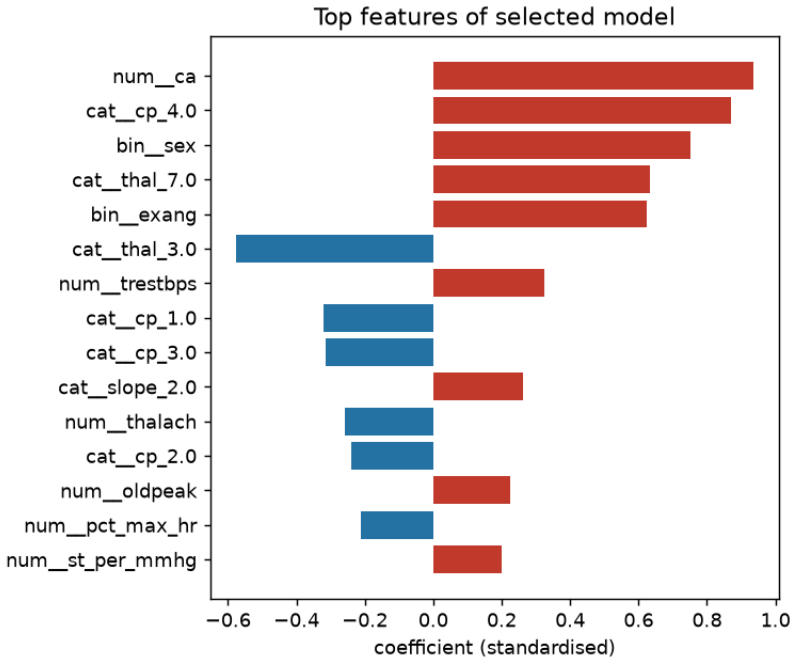


Figure 9. Largest standardised coefficients of the selected model. Red raises the risk.

The largest coefficients are the number of coloured vessels (+0.94), asymptomatic chest pain (+0.87), male sex (+0.75), a reversible thallium defect (+0.63) and exercise angina (+0.62); a normal thallium scan lowers risk (-0.58). This matches the EDA.

5. Experiment tracking (MLflow)

heart.train delegates logging to a small RunTracker interface. heart.tracking.MlflowTracker implements it, and --no-mlflow swaps in a no-op tracker for unit tests. Each candidate becomes one run in the experiment heart-disease-classifier, which logs the best grid parameters, the CV mean/std and test metrics for all five scores, the ROC and confusion-matrix PNGs, and the fitted pipeline (mlflow.sklearn.log_model with an input example). The winning run is tagged selected=true together with the written rationale.

Implementation notes. MLflow 3.17 rejects the plain ./mlruns file backend by default, so run metadata is stored in a local SQLite file (mlruns/mlflow.db) and artifacts in mlruns/artifacts/. Everything stays local, with no server. MLflow 3 also stores sklearn models in the skops format, which only loads allow-listed types; the tracker allows exactly the flagged types that come from the heart, sklearn and numpy packages.

Table 5. Runs currently in the tracking store (3).

run name	status	selected
logreg	FINISHED	true
random_forest	FINISHED	false
hist_gb	FINISHED	false

```
mlflow ui --backend-store-uri sqlite:///mlruns/mlflow.db   # http://127.0.0.1:5000
```

6. Packaging and reproducibility

- **Model artefacts.** The complete fitted pipeline (imputation, derived features, encoding/scaling and the classifier) is saved as models/model.joblib, which the API loads, and as models/mlflow_model/ via mlflow.sklearn.save_model, together with MLmodel, conda.yaml, requirements and an input example. models/model_meta.json records the chosen model, its parameters, metrics and selection rationale.
- **Environment.** requirements.txt pins every direct dependency to the version verified in the project venv (Python 3.12.10; e.g. scikit-learn 1.9.1, pandas 3.0.6, mlflow 3.17.1). requirements-serve.txt is the smaller runtime set used by the Docker image.
- **Code layout.** An installable src/ package (pip install -e . via pyproject.toml). config.py is the single source of truth for paths, column roles and the seed; data, features, models, evaluate, train, tracking, predict, api and observability each do one job.
- **Determinism.** A fixed seed (523) is used for the split, the CV shuffling and the models. Re-running training reproduces the table in Section 4.

```
py -3.12 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -m pip install -e .
.\.venv\Scripts\python.exe scripts\download_data.py
.\.venv\Scripts\jupyter.exe nbconvert --to notebook --execute --inplace notebooks\01_eda.ipynb
.\.venv\Scripts\python.exe -m heart.train
```

7. Automated testing and CI/CD

The pytest suite currently reports 23 passed, and ruff: all checks passed. It covers:

- **Data:** target binarisation, '?' parsed as missing, rows kept, out-of-domain codes nulled, missing-column error, stratified split ratio.
- **Features:** derived-feature arithmetic, output shape and finiteness after imputation, unseen categories ignored, numeric columns standardised.
- **Model:** pipeline fit/predict_proba, the three model families and their grids, the selection rule (recall within the AUC band), and an end-to-end quick training run on the real data with an AUC floor of 0.75.
- **API:** predict function, /health (ok and degraded), /predict (valid input, missing ca/thal, 422 on invalid input), /metrics content, and the JSON request-log format.

.github/workflows/ci.yml runs on push, pull request and manual dispatch. Its steps are checkout, setup-python 3.12 with pip cache, install requirements and the package, ruff (GitHub annotations), download the data, pytest with JUnit XML, full training with the log captured, and upload of models/, reports/ and mlruns/ as a per-run artifact even if an earlier step failed. Any failing step fails the job.

Status: The workflow ran on GitHub after the push (run #2, job lint-test-train) and passed in 1m 52s with one uploaded artifact.

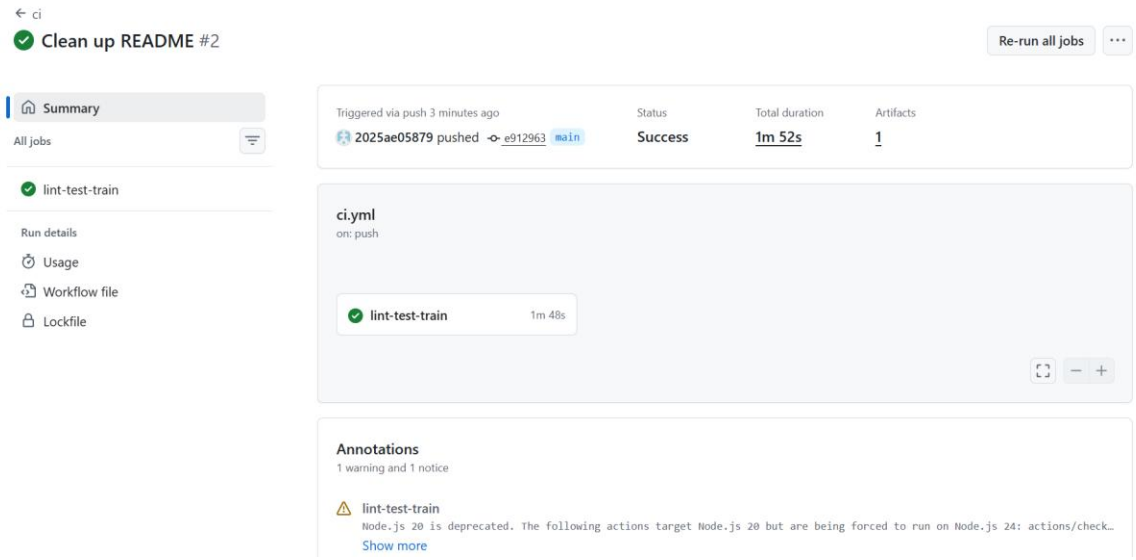


Figure 10. Screenshot - GitHub Actions run page for ci.yml with all steps green and the uploaded artifact listed

8. Containerisation

The Dockerfile has two stages on python:3.12-slim. The builder creates /opt/venv, installs requirements-serve.txt and then the heart package (without dev tools). The runtime stage copies only that venv plus models/model.joblib, runs as uid 10001, sets HEART_MODEL_PATH, declares a HEALTHCHECK against /health, and starts uvicorn on port 8000. .dockerignore keeps .venv, mlruns, data, notebooks and tests out of the build context.

The API contract: POST /predict accepts the 13 raw fields as JSON (ca and thal are optional; ranges are validated by pydantic, and invalid input returns 422) and returns prediction, label, probability_disease and confidence = max(p, 1 - p). GET /health reports model status, version and pod.

```
POST /predict  samples/sample_request.json
-> {"prediction":1,"label":"heart disease","probability_disease":0.9671,"confidence":0.9671}
POST /predict  samples/sample_request_low_risk.json
-> {"prediction":0,"label":"no heart disease","probability_disease":0.0243,"confidence":0.9757}
```

```
C:\Users\jsaish.pamarti>docker images heart-api

IMAGE              ID             DISK USAGE   CONTENT SIZE   EXTRA
heart-api:0.2.0    80c193c64739        676MB          155MB
```

Figure 11. Screenshot - terminal: end of `docker build -t heart-api:0.2.0 .` and `docker images heart-api`

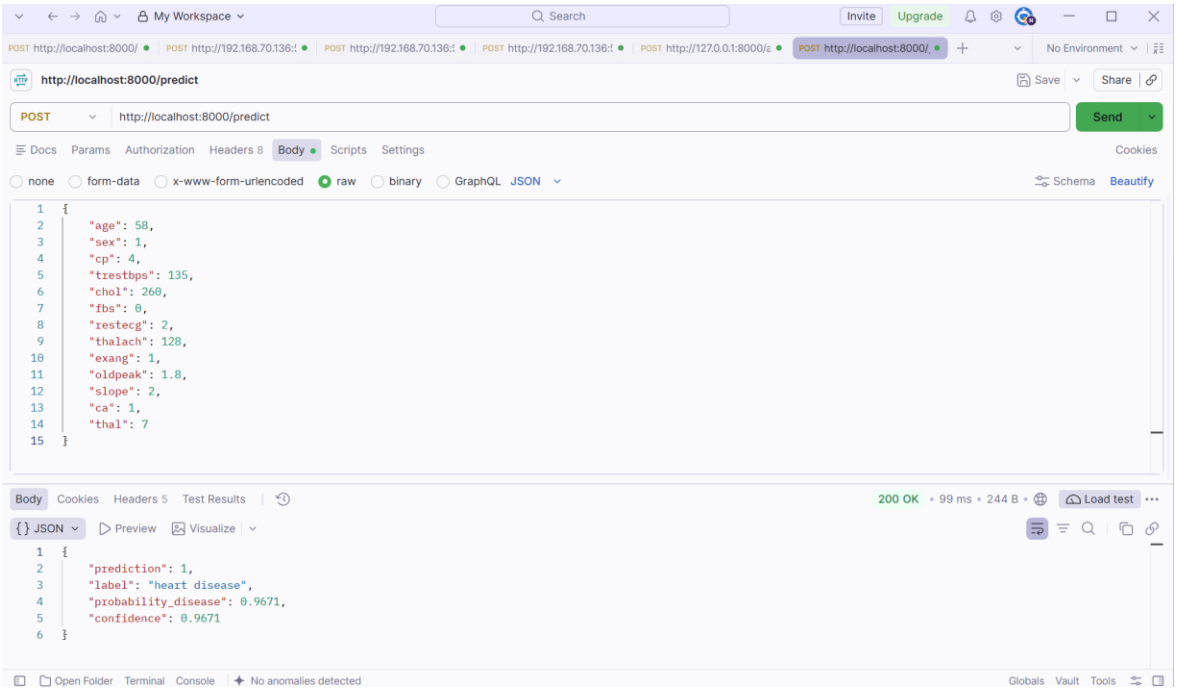


Figure 12. Screenshot - terminal: `docker run -d -p 8000:8000 heart-api:0.2.0`, then curl /health and curl /predict with samples/sample_request.json

9. Production deployment on Kubernetes

The target is the Docker Desktop Kubernetes cluster (context docker-desktop, node desktop-control-plane, v1.36.1). k8s/deployment.yaml runs 2 replicas of heart-api:0.2.0 with imagePullPolicy IfNotPresent, so the locally built image is used. It defines readiness (every 5 s) and liveness (every 15 s) probes on /health, requests of 100m CPU / 192Mi and limits of 500m / 384Mi, runs as non-root, and uses a RollingUpdate with maxUnavailable 0. The image went from 0.1.0 to 0.2.0 this way with no downtime. k8s/service.yaml exposes the pods through a LoadBalancer service.

Port-80 conflict. The first deployment used service port 80. localhost:80 on the laptop is already published by an unrelated container (another local web application), so Docker Desktop's load-balancer proxy could not bind it. kubectl describe svc showed SyncLoadBalancerFailed, and http://localhost/ returned the other application's page. The service now uses port 8080 -> 8000, and http://localhost:8080 is served by both replicas (/predict requests appear in both pods' logs and metrics).

```
kubectl apply -f k8s/
kubectl rollout status deploy/heart-api
kubectl get pods,svc,deploy -o wide
curl http://localhost:8080/health
```

```
C:\Users\jsaish.pamarti>kubectl get deployments,pods,svc -l app.kubernetes.io/name=heart-api
NAME                           READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/heart-api      2/2     2            2           42h

NAME                                READY   STATUS    RESTARTS   AGE
pod/heart-api-57977d849-4rrhv       1/1     Running   0          42h
pod/heart-api-57977d849-tbw9g       1/1     Running   0          42h

NAME                 TYPE           CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
service/heart-api    LoadBalancer   10.96.193.242   172.20.0.5    8080:30724/TCP   42h
```

Figure 13. Screenshot - terminal: `kubectl get pods,svc,deploy -o wide` showing 2/2 pods Running, the LoadBalancer on 8080 and image 0.2.0

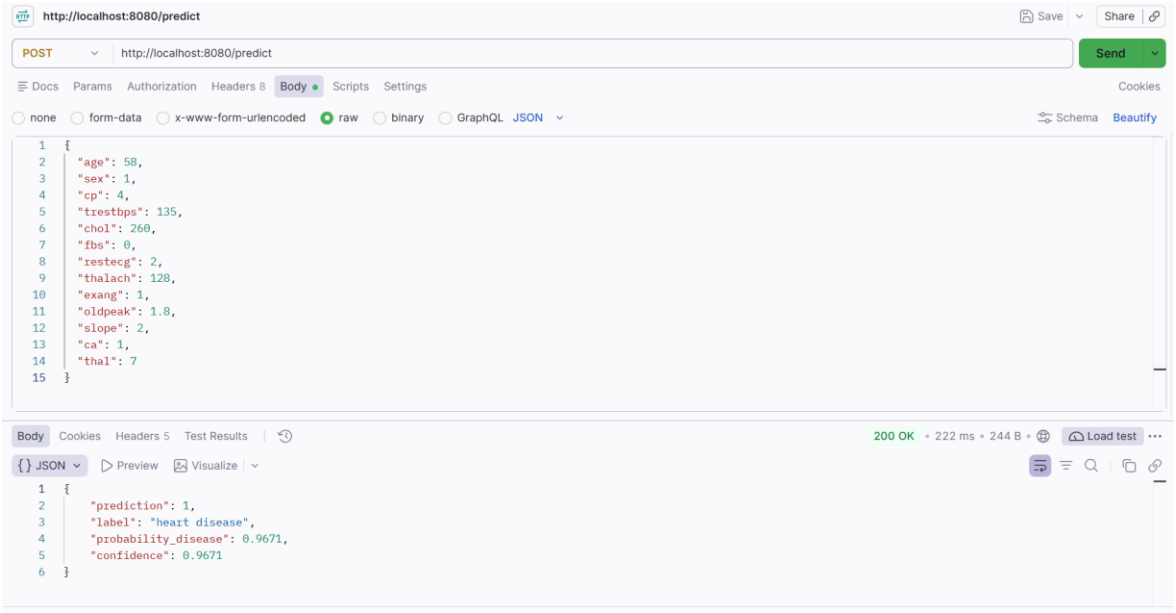


Figure 14. Screenshot - browser: http://localhost:8080/docs with /predict executed using the sample body and a 200 response

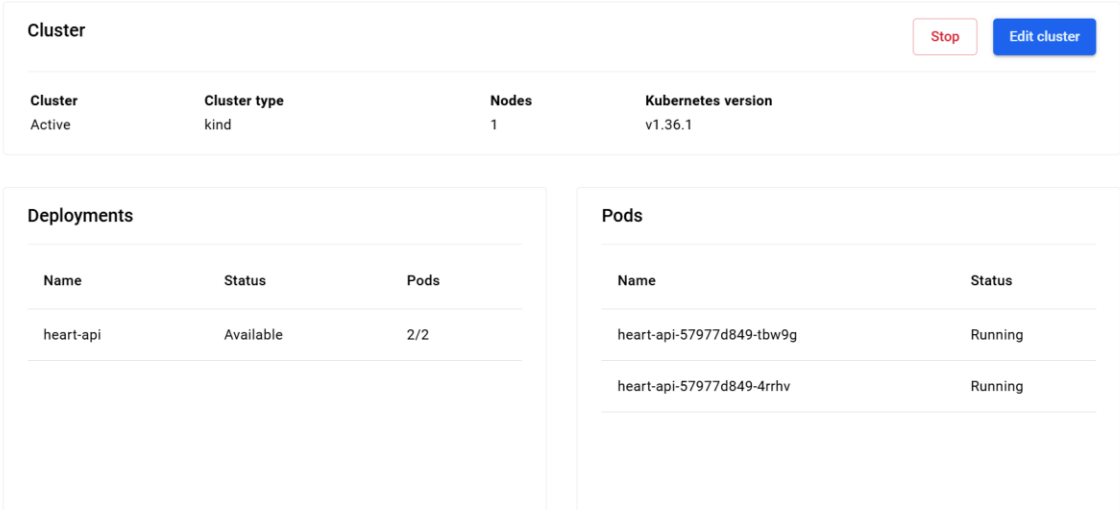


Figure 15. Screenshot - Docker Desktop Kubernetes view with the cluster running and the heart-api pods

10. Monitoring and logging

- **Request logging.** An HTTP middleware writes one JSON line per request to stdout with ts, pod, request_id (also returned as the x-request-id header), method, path, status, latency_ms, client, prediction and probability. Kubernetes collects these, and kubectl logs -l app.kubernetes.io/name=heart-api --prefix shows them.
- **Metrics.** /metrics uses prometheus_client and exposes heart_api_requests_total {method, path, status}, the heart_api_request_duration_seconds histogram, heart_api_predictions_total{label}, a histogram of the predicted probability, and heart_api_model_loaded. Every series carries a pod label. Path labels use the route template, which keeps label cardinality bounded.

- **Stack.** monitoring/docker-compose.yml runs Prometheus 3.5 (scraping host.docker.internal:8080 every 5 s) and Grafana 12.1, with the datasource and the dashboard provisioned from files. The dashboard 'Heart API - model service health' has headline stats (model loaded, total predictions, share predicted positive compared with the 46% training base rate as a drift hint, error rate, p95 latency) and time series for request rate, p50/p95/p99 latency, error share by status, predictions per class, requests per replica, and the predicted-risk distribution.
- **Verification.** After about 120 requests from scripts/load_test.py (real patient rows plus ~6% deliberately invalid payloads), the Prometheus target was UP, queries returned per-class prediction counts, 200/422 status counts and a p95 /predict latency of about 25 ms. Grafana's API reported the datasource healthy and the dashboard present.
- **Limitation.** Prometheus reaches the pods through the LoadBalancer and reuses one keep-alive connection, so most scrapes land on the same replica. The pod label keeps the replicas' counters as separate series, but per-replica totals are incomplete. In a real cluster, Prometheus would run inside it with kubernetes_sd_configs; the pods already carry prometheus.io/* annotations for that.



Figure 16. Screenshot - terminal: `kubectl logs -l app.kubernetes.io/name=heart-api --prefix --tail=20` with JSON lines for /predict

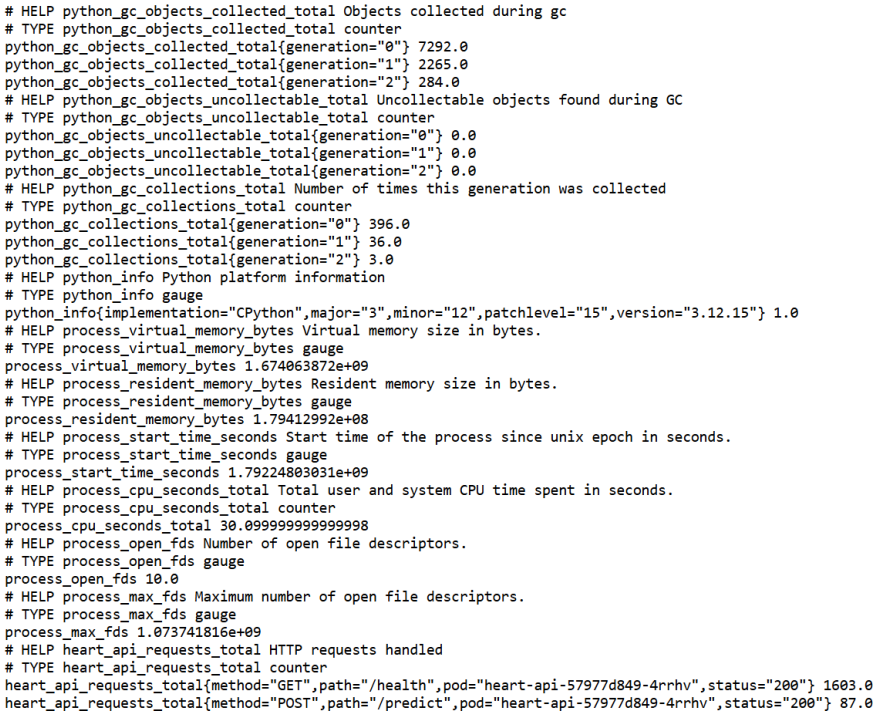


Figure 17. Screenshot - browser: http://localhost:8080/metrics showing the heart_api_* series

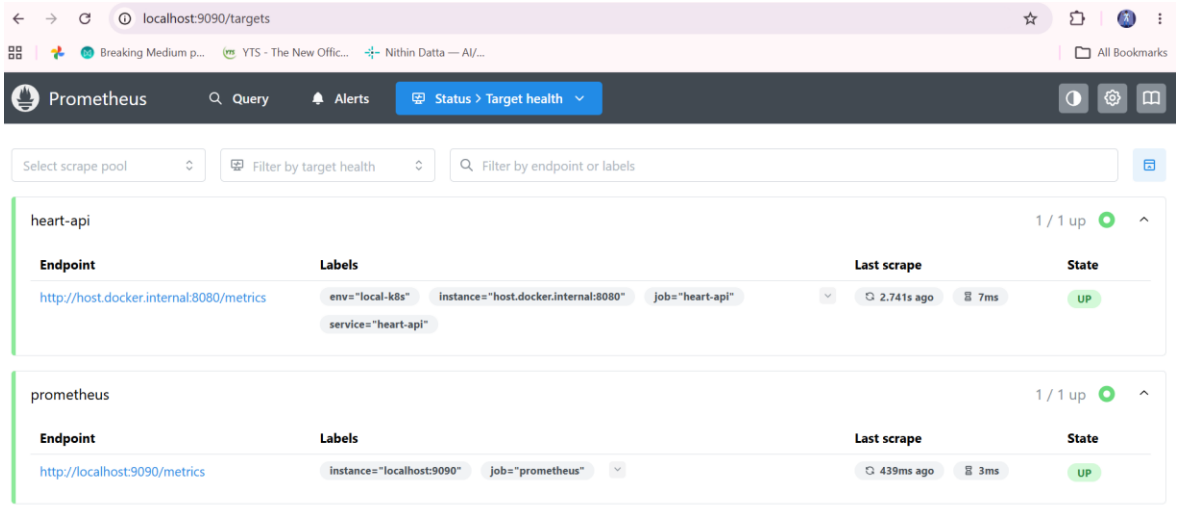


Figure 18. Screenshot - browser: http://localhost:9090/targets with the heart-api target UP

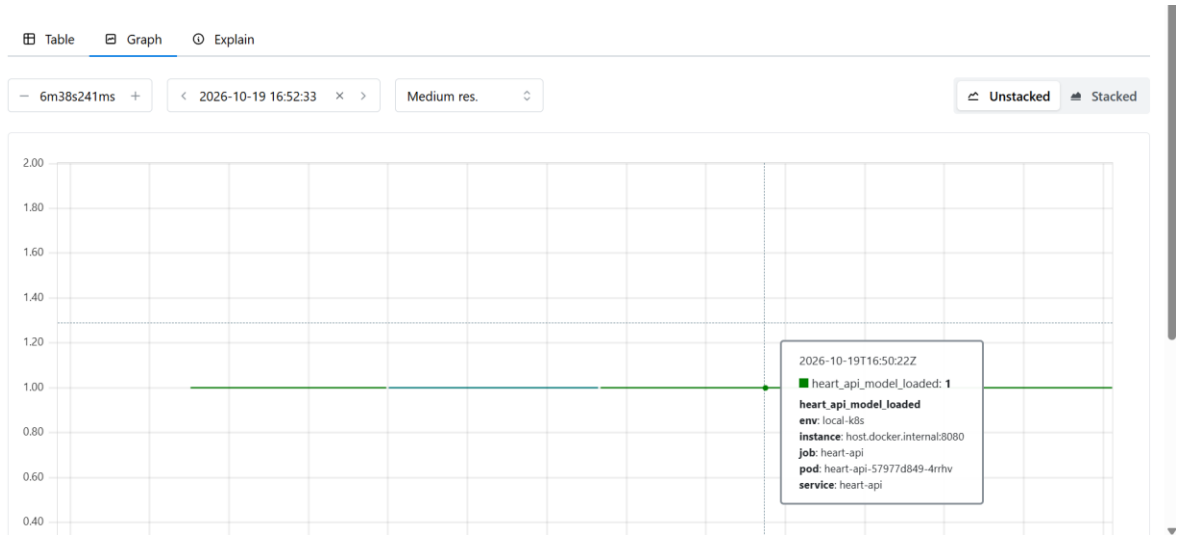


Figure 19. Screenshot - browser: Prometheus query `sum by (label) (heart_api_predictions_total)` in graph view

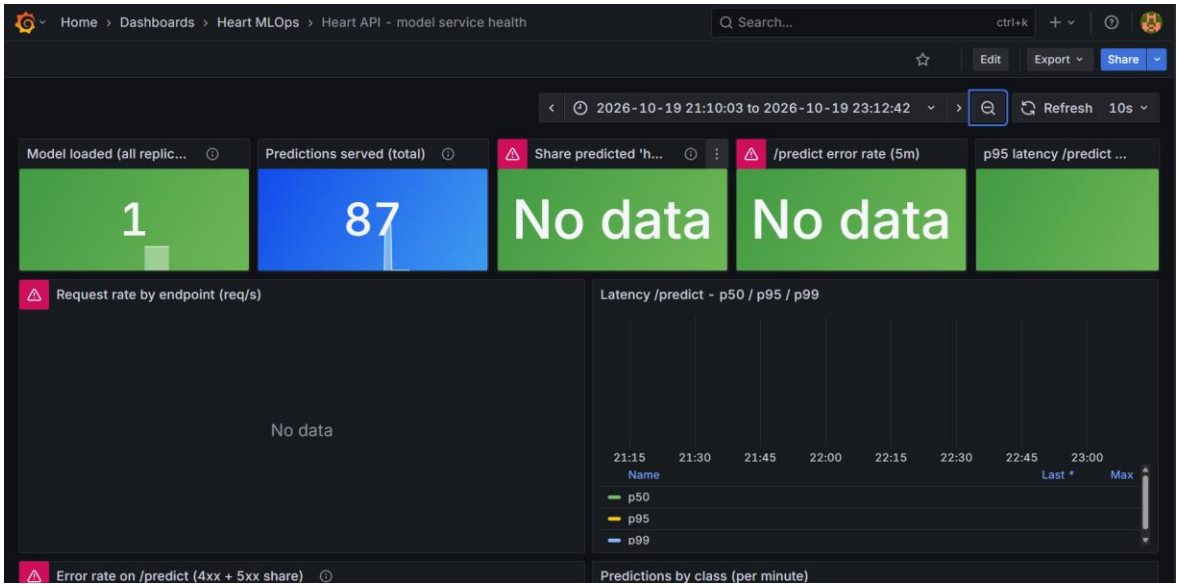


Figure 20. Screenshot - browser: http://localhost:3000/d/heart-api-ops after a fresh load test, last 15 minutes, all panels populated

11. Limitations and next steps

- 303 patients from one hospital (Cleveland, 1988) is a small, dated, referral-biased sample. The metrics will not transfer to general screening without external validation, for example on the Hungarian, Swiss and Long Beach UCI subsets.
- The 0.5 threshold is not tuned. For screening, a lower threshold chosen on CV to hit a target recall (for example 0.90) would better match the cost asymmetry, and the probabilities should be calibration-checked.
- There is no CD step yet. Next steps are to add an image build-and-push job and deploy by tag.
- Monitoring tracks service health and the prediction mix, not data drift on the inputs. A per-feature drift check against the training distribution (PSI or KS) would be the next metric to add.
- Run Prometheus inside the cluster with pod discovery to fix the single-connection scrape bias. Add an Ingress and HPA if the service ever needed real traffic.

12. Repository and deliverables

Repository link: https://github.com/2025ae05879/mlops-heart-disease

Table 6. Repository map.

path	content
scripts/download_data.py, data/	data acquisition + cleaned dataset
notebooks/01_eda.ipynb, reports/figures/	EDA (executed) and all figures
src/heart/	pipeline, training, tracking, prediction, API, observability
reports/model_comparison.{csv,md}	model comparison
models/	model.joblib, mlflow_model/, model_meta.json
tests/, .github/workflows/ci.yml	pytest suite and CI workflow
Dockerfile, requirements*.txt	container image and pinned dependencies
k8s/	Deployment + LoadBalancer Service
monitoring/	Prometheus + Grafana (compose, config, dashboard JSON)

screenshots/, screenshots/proof/	screenshots and saved command outputs